

OpenZeppelin Fixed Point Math Diff Audit



June 2, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Notes & Additional Information	7
N-01 Internal Precision Comment Overstates raw_log2 Error Bound	7
N-02 log10 Underestimates by One Ulp at Integer Powers of Ten	7
N-03 UD30x9 Logarithm Error Name Conflates Undefined Input with Unrepresentable Result	8
N-04 Code and Documentation Improvements	9
Conclusion	11
Appendix	12
Issue Classification	12

Summary

Type	Library	Total Issues	4 (4 resolved)
Timeline	From 2026-05-26 To 2026-05-28	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	4 (4 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin conducted a [diff audit](#) of the [OpenZeppelin/contracts-sui](#) repository covering the changes between base commit [af09be7](#) and target commit [55a7971](#).

In scope were the following modified files:

```
math
├── fixed_point
│   └── sources
│       ├── internal
│       │   └── common.move
│       ├── sd29x9
│       │   ├── conversions.move
│       │   ├── sd29x9_base.move
│       │   └── sd29x9.move
│       └── ud30x9
│           ├── conversions.move
│           ├── ud30x9_base.move
│           └── ud30x9.move
```

System Overview

`openzeppelin_fp_math` is a Move library for the Sui blockchain that provides decimal fixed-point arithmetic at 9 decimals of precision, matching Sui's native coin decimals. A raw value of `1_000_000_000` represents the real number `1.0`. The library exposes two principal numeric types stored in a single `u128`: the unsigned `UD30x9` and the signed `SD29x9`. The pre-existing arithmetic, comparison, rounding, bitwise, and cross-type conversion API, the cast-versus-convert distinction, and the `SD29x9` two's-complement representation are documented in the [prior fp-math audit](#) and are not re-described here.

The internal helpers shared by both numeric types live in `common`. The diff under audit extends this module with the routines that drive every new logarithm: the `ln(2)` and `log10(2)` factor constants at scale `10^18`, the `raw_log2` kernel that computes base-2 logarithm at scale `10^18` via input normalization plus an iterative squaring loop, and the `apply_log2_factor` helper that converts a `raw_log2` magnitude into a base-e or base-10 result at the user-facing scale. The package depends on `openzeppelin_math` for unsigned-integer building blocks: `u128::mul_div`, `u128::log2`, `u128::msb`, `u256::sqrt`, and the `rounding` rounding-mode helpers.

Within `math/fixed_point`, the diff touches seven source files and adds the following:

- **Square root:** `sqrt` is introduced on both `UD30x9` and `SD29x9`, with the implementations living in `ud30x9_base` and `sd29x9_base`. Both implementations promote the raw value to `u256`, multiply by the scale factor to preserve precision, and delegate to the `u256::sqrt` helper with round-down rounding before re-wrapping. The signed variant aborts with the new `ENegativeSqrt` constant when the input is negative.
- **Logarithms:** `log2`, `ln`, and `log10` are introduced on both types. All three operations route through the shared `raw_log2` kernel; `ln` and `log10` then apply a base-conversion factor (`ln2_e18`, `log10_2_e18`) via `apply_log2_factor`. The `UD30x9` variants reject inputs strictly below one, since the result would be negative and unrepresentable. The `SD29x9` variants reject zero and negative inputs. A new `ELogUndefined` constant is added to each type's base module.
- **Method-call-style refactor:** A subset of pre-existing operations is rewritten to chain through receiver-syntax (for example, `result.wrap_components()` in place of `wrap_components(result)`). These rewrites do not change semantics.

- **Method export aliases:** New `public use fun` declarations expose `sqrt`, `log2`, `ln`, and `log10` as receiver-syntax methods on `UD30x9` and `SD29x9`.

Security Model and Trust Assumptions

The package remains a pure-math library across this diff. It defines no shared objects, capabilities, dynamic fields, transfer policies, or upgrade entry points, reads no Sui system primitives such as `Clock` or `Random`, and introduces no privileged roles. The pre-existing security posture, including the `-2^127` singularity on `SD29x9::negate` and `SD29x9::abs`, is documented in the prior fp-math audit. This section enumerates the trust assumptions that the diff newly introduces.

Library Consumers: The new public operations follow the same precondition-and-abort pattern as the pre-existing arithmetic. The diff adds the following aborts:

- `SD29x9::sqrt` aborts on negative inputs.
- `SD29x9` logarithms abort on zero and negative inputs.
- `UD30x9` logarithms abort on inputs strictly below one.

Pre-existing aborts on conversion-range violations, zero divisors, and shift-out-of-range continue to apply unchanged.

Numerical Precision: The new logarithm pipeline applies round-down floors at multiple stages. The kernel right-shifts inputs before lifting to scale `10^18`, and the per-base conversion factor itself is floored. The new `sqrt` operations round down. Each new operation documents its own rounding direction and precision tolerance in its docstring, and callers are expected to honor that contract.

External Dependency: The new operations depend on the correctness of `openzeppelin_math` for `u256::sqrt`, `u128::log2`, `u128::msb`, `u128::mul_div`, and the `rounding` helpers. The Sui Move runtime is assumed to enforce native integer overflow and divide-by-zero aborts on those primitives.

Notes & Additional Information

N-01 Internal Precision Comment Overstates `raw_log2` Error Bound

The `raw_log2` kernel that powers `log2`, `ln`, and `log10` on both `UD30x9` and `SD29x9` normalizes the input to `[1, 2)` while tracking the integer part `n`, lifts the residue to the internal `1018` scale, then recovers the fractional bits through an iterative squaring loop. The error-analysis comment block on the `INTERNAL_LOG_SCALE` constant claims the total `frac` error stays under `~103` at scale `1018` and is therefore well below one user-facing ulp at scale `109`. That analysis only covers the squaring loop and overlooks the lossy `x_raw >> n` right-shift performed during input normalization, which discards up to `n` low-order bits before the lift to `1018` and dominates the kernel's error. Empirically the deficit grows with `n`: `raw_log2(u128::MAX)` lands two user ulps low at scale `109` (returning `98_102_647_144` against a 200-digit-precision reference of `98_102_647_146`), while `raw_log2(1_234_567_123_456_789)` lands one user ulp low and a control input of `3.0` deviates by only `~15` at scale `1018`. The kernel's actual bound is `≤ 2` user ulps, monotone-down, and the single-ulp guarantees asserted in the `UD30x9::ln` and `UD30x9::log10` docstrings inherit the same defect and should be revisited under the same bound.

Consider rewording the precision note to state that the kernel result is at most two user-facing ulps below the true value, monotone-down, with the error dominated by the `x_raw >> n` pre-loop truncation rather than the squaring loop and growing with `n` (the integer part of `log2`), and separating the unrelated justification for the `1018` internal scale (headroom for inputs near `1.0`) so each point reads independently.

Update: Resolved in [pull request #333](#).

N-02 `log10` Underestimates by One Ulp at Integer Powers of Ten

Both `UD30x9::log10` and `SD29x9::log10` derive their result via the identity `log10(x) = log2(x) * log10(2)`, with both factors floored. Consequently `log10(10k)` returns `k *`

`SCALE - 1` rather than `k * SCALE` for every integer `k >= 1`; for example, `log10(1_000)` yields `2.999_999_999` instead of `3.000_000_000`. The deviation is bounded by one user ulp, but the affected set is precisely the integer lattice (`10`, `100`, `1_000`, ...) that pricing-tier and tick-decoding consumers most commonly key on; reporting `2.999_999_999` where the natural answer is `3` is liable to trip equality checks, rounding boundaries, and decimal-place tagging downstream. The defect is specific to `log10` and `ln` at round inputs: `log2` at powers of two emerges exact as a side effect of the squaring loop, and only `log10` admits an exact remedy, since `ln(10^k) = k * ln(10)` is irrational and offers no representable target to round to.

Consider adding an exact fast path that detects power-of-ten inputs using the existing `u128::is_power_of_ten` helper and recovers the integer exponent through the `u256::log10` kernel. For a `UD30x9` raw value `x`, the result is `(u256::log10(x, down) - 9) * SCALE`; the `-9` offset removes the user scale already embedded in `x`. For `SD29x9`, the fast path must execute on the decomposed magnitude before the sub-unit sign branch, so that inputs of the form `10^-k` (whose magnitudes are themselves powers of ten below `SCALE`) return exact `-k * SCALE` rather than the current biased magnitude. The docstring on `UD30x9::log10` claiming `log10(10) == one() - 1 ulp` should also be updated to reflect the exact result.

Update: Resolved in [pull request #344](#).

N-03 UD30x9 Logarithm Error Name Conflates Undefined Input with Unrepresentable Result

The `UD30x9` module defines a single error constant `ELogUndefined` with the message "Logarithm is undefined: input must be greater than or equal to one", and raises it from `ln`, `log10`, and `log2` whenever the input falls below one. The name and message conflate two distinct failure modes: `log(0)` is genuinely undefined in real analysis, but `log(x)` for any `x` in `(0, 1)` is a well-defined negative real number that simply cannot be represented in the unsigned `UD30x9` type. Callers reading the abort are told that their input is mathematically invalid when, for inputs in `(0, 1)`, the actual constraint is a representation limit of the chosen type. The distinction matters for consumers deciding whether to switch to `SD29x9` (whose `ELogUndefined` is scoped to zero and negative inputs and carries the message "input must be strictly positive"), clamp the input, or treat the call as a programming bug.

Consider splitting the `UD30x9` error into two distinct codes: one preserving the existing `ELogUndefined` semantics for the truly undefined input `x == 0` (matching `SD29x9`), and a separate code such as `ELogResultUnrepresentable` for inputs in `(0, 1)` whose result

would be negative. The latter should carry a name and message that signal type capacity rather than mathematical pathology, for example "Logarithm result is negative and not representable in UD30x9; use SD29x9 if negative results are needed". The `Aborts` sections of the `ln`, `log10`, and `log2` docstrings should be updated to mention the new code so callers can branch on the cause without inspecting the input themselves.

Update: Resolved in [pull request #341](#).

N-04 Code and Documentation Improvements

Throughout the codebase, several opportunities to improve readability, consistency, and self-documentation were identified:

- The constants `INTERNAL_LOG_SCALE` and `LOG_FACTOR_DENOM_E27` are declared inside the `// === Package Functions ===` block of `common.move` rather than in a dedicated `// === Constants ===` section. This breaks the section ordering convention applied throughout the rest of the codebase. Consider moving both declarations into a separate `// === Constants ===` section placed at the top of the module, alongside the other constant definitions.
- The `div_away_u256` function of `common.move` aborts through the native integer division semantics when `denominator` is zero, yet its docstring does not include a `#### Aborts` block. Every other abort-bearing function in the audit scope documents this behavior explicitly. Since the helper is declared `public(package)`, an in-package caller could overlook the precondition. Consider adding an `#### Aborts` block stating that the function aborts when `denominator` is zero.
- The `sqrt` function of `sd29x9_base.move` documents that it "aborts if `x` is negative" without naming the underlying error constant `ENegativeSqrt`. In contrast, the sibling functions `ln`, `log10`, and `log2` all name `ELogUndefined` explicitly in their `#### Aborts` blocks. Consider updating the `sqrt` docstring to reference `ENegativeSqrt` for consistency.
- In the arithmetic `public use fun` block of `sd29x9.move`, `sqrt` is placed between `pow` and `rem`, breaking the alphabetical ordering followed by the rest of the block. The analogous block in `ud30x9.move` preserves the correct order. Consider relocating the `sqrt` declaration to its alphabetical position to keep the two modules aligned.

Consider applying the changes described above to improve code consistency, readability, and self-documentation.

Update: Resolved in [pull request #332](#).

Conclusion

The `openzeppelin_fp_math` package implements decimal fixed-point arithmetic at 9 decimals of precision on Sui Move, exposing unsigned and signed 128-bit types together with arithmetic, comparison, conversion, casting, and bitwise helpers. OpenZeppelin audited the diff across the seven source files of `math/fixed_point`, focusing on the recent square-root and logarithm additions to the type-base modules and the supporting kernel additions in the shared helpers module.

The code in scope is small, carries no cross-cutting protocol concerns, and is exercised by a test suite that pins reference values at known boundary inputs for the logarithm pipeline and at exact powers of two for `log2`. Reasoning about correctness is concentrated in the integer kernels: the bit-shifting normalization and squaring loop inside `raw_log2`, the `mul_div` chain in `apply_log2_factor`, and the truncating `u256::sqrt` integration in both `sqrt` implementations. These kernels carry inline comments documenting their precision claims and rely on `u256` widening to avoid `u128` overflow at the intermediate steps.

No critical, high, medium, or low severity issues were identified. The remaining findings are concentrated in documentation and code-quality polish, covering internal precision documentation, error-message naming and clarity, precision behavior at identity inputs, and section ordering and docstring consistency.

The OpenZeppelin development team is thanked for their responsiveness and willingness to provide technical context throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.